Digital Library Management System — Interview Guide

20 Comprehensive Technical Interview Questions & Answers | Spring Boot, Spring AI, React, Security & RAG

1. General Architecture & Overview

Q1: Can you provide a high-level overview of this project?

Answer: This project is an AI-powered Full-Stack Digital Library Management System. Beyond traditional library operations like catalog management, book issuance, return tracking, and automated late fine calculation, it features an intelligent RAG (Retrieval-Augmented Generation) assistant. Students can chat in natural language to discover books, check inventory, and receive recommendations based strictly on real-time library data.

Q2: What is the technology stack used in this application?

Answer: The backend is built with Spring Boot 3, Spring Security, Spring Data JPA, and Spring AI. The database layer uses MySQL for relational persistence and an in-memory VectorStore for semantic vector embeddings. The frontend is built using React with Vite and styled using custom modern CSS. The AI capabilities are powered locally using Ollama running the qwen3 LLM and nomic-embed-text embedding model.

2. Generative AI & RAG (Retrieval-Augmented Generation)

Q3: What is RAG, and why did you use it instead of asking a normal LLM directly?

Answer: A public or raw LLM doesn't know private internal library records and is prone to hallucinations—it might invent non-existent book titles or quantities. RAG solves this by retrieving exact, relevant library records from our database first, injecting them into the LLM's system prompt as verified context, and constraining the model to answer solely using that context.

Q4: How did you implement the RAG pipeline in Spring AI?

Answer: Whenever books are loaded on startup or updated, their metadata (Title, Author, Category, ISBN, and stock) is converted into text documents. Using nomic-embed-text, these documents are embedded into dense vector representations in a VectorStore. When a user asks a question, Spring AI performs a cosine similarity search (topK=4), extracts the most relevant book snippets, passes them into ChatClient with a strict system prompt, and returns the verified answer.

Q5: What happens to the AI vector store when an Admin adds, updates, or deletes a book?

Answer: We maintain data consistency between the relational database and the vector store. To prevent race conditions or dirty state if a database transaction rolls back, we register a synchronization hook using TransactionSynchronizationManager.registerSynchronization() inside the @Transactional boundary. The vector index update or deletion is executed in the afterCommit() callback only after the MySQL transaction has been committed.

3. Authentication & Security

Q6: How did you implement security and authentication in this project?

Answer: We implemented stateless authentication using JSON Web Tokens (JWT) and Spring Security 6. We use an HMAC-SHA256 signature with a minimum 256-bit secret key. Every incoming request is intercepted by our custom JwtAuthenticationFilter, which parses the Authorization: Bearer header, validates the signature and expiration, extracts user claims and roles, and populates the SecurityContextHolder.

Q7: How are roles and authorization handled between Students and Admins?

Answer: We enforce Role-Based Access Control (RBAC) using @EnableMethodSecurity and URL matchers. Public read endpoints like /api/books are open to everyone. Sensitive member endpoints require authenticated tokens, while catalog management (adding, updating, deleting books) and issuing loans require the ADMIN role. If a Student attempts an admin action, Spring Security throws an AccessDeniedException, which our GlobalExceptionHandler converts into an HTTP 403 Forbidden response.

Q8: How do you store user passwords securely in the database?

Answer: We use Spring Security's BCryptPasswordEncoder bean. Passwords are never stored or transmitted in plain text. BCrypt incorporates a random salt and adaptive work factor, protecting passwords against rainbow-table and brute-force attacks. Furthermore, in our UserController, the password field is explicitly excluded from API responses.

4. Concurrency & Business Logic

Q9: How do you prevent race conditions when multiple students try to borrow the last remaining copy of a book?

Answer: If two requests arrive simultaneously for a book with availableQuantity = 1, a naive read-then-write could cause an oversell. To prevent this, we implemented Pessimistic Locking using @Lock(LockModeType.PESSIMISTIC_WRITE) in the BookRepository (findByIdForUpdate). This executes a SQL SELECT ... FOR UPDATE, placing an exclusive database row lock so only one transaction can deduct inventory at a time.

Q10: How does the fine calculation logic work for overdue books?

Answer: In FineService, when an issued book is returned, we calculate the overdue duration using ChronoUnit.DAYS.between(dueDate, returnDate). If lateDays > 0, we multiply lateDays by a fixed daily rate (BigDecimal('10.00')) and persist a new Fine record with status UNPAID. If the book was returned on time, no fine is created and the API returns HTTP 204 (No Content).

Q11: Why did you choose BigDecimal instead of double or float for financial/fine calculations?

Answer: In Java, primitive floating-point types (float and double) use binary IEEE 754 representations, which suffer from rounding errors in decimal arithmetic. For monetary calculations like late fees, BigDecimal provides exact precision and scale control, preventing compounding financial discrepancies.

Q12: Can an admin delete a book that is currently on loan?

Answer: No. In BookService.deleteBook(), we verify whether any active issues exist by calling issueRepository.existsByBookIdAndStatusIgnoreCase(id, 'ISSUED'). If copies are currently on loan, the service rejects the request with an IllegalArgumentException to maintain referential and physical integrity.

5. Backend REST & Exception Handling

Q13: How do you handle errors and exceptions across the backend?

Answer: We use a centralized @RestControllerAdvice class (GlobalExceptionHandler). Instead of leaking raw Java stack traces to the frontend, it catches specific exceptions like AccessDeniedException (HTTP 403), MethodArgumentNotValidException (HTTP 400 with a map of field validation messages), and general runtime errors (HTTP 400 or 404), returning consistent JSON error payloads.

Q14: How is server-side input validation implemented?

Answer: We use Jakarta Validation (spring-boot-starter-validation). Entity fields and DTOs are decorated with annotations such as @NotBlank, @Email, and @Size(min = 8) for passwords. In controller request mappings, @Valid triggers automated validation before method execution, blocking malformed data at the HTTP layer.

6. Database & ORM

Q15: What is the difference between JPA and Hibernate?

Answer: JPA (Jakarta Persistence API) is a standard specification defining object-relational mapping interfaces in Java. Hibernate is the underlying implementation that provides the actual engine, SQL generation, caching mechanisms, and connection handling.

Q16: Why did you set spring.jpa.open-in-view=false in production configuration?

Answer: By default, Open-Session-In-View (OSIV) keeps database connections open during the entire HTTP request lifecycle, including view rendering. Disabling it ensures that database connections are borrowed and released strictly within the service transaction boundaries, preventing connection pool exhaustion under high traffic.

7. Frontend (React & Architecture)

Q17: Why did you choose Vite over Create React App (CRA)?

Answer: Create React App uses Webpack and is now deprecated. Vite leverages native ES Modules (ESM) in development and Rollup for production builds, providing near-instantaneous hot module replacement (HMR), extremely fast startup times (< 1 second), and significantly smaller production bundles.

Q18: How is client-side state and token persistence managed in the frontend?

Answer: We use React state (useState, useEffect, useRef) for local UI state. The authenticated user profile and JWT token are persisted in browser sessionStorage. For every authenticated API request, an apiRequest utility attaches the bearer token in the Authorization header. If an HTTP 401 or token expiration occurs, custom events trigger a clean sign-out and session reset.

8. Testing & Code Quality

Q19: How did you test your business logic?

Answer: We wrote isolated unit tests using JUnit 5 and Mockito (e.g., FineServiceTest and BookServiceTest). By mocking repository and AI dependencies, we verify edge cases—such as on-time returns returning null, multi-day overdue fine calculations, unauthorized user access, and invalid quantity invariants—without requiring an external database or network calls.

Q20: How are automated integration tests run without interfering with the live MySQL database?

Answer: We configured a dedicated src/test/resources/application.properties profile that automatically uses an in-memory H2 Database (jdbc:h2:mem:testdb) with ddl-auto=create-drop. When running mvn test, all unit and Spring application context tests execute quickly and independently, ensuring continuous integration builds remain reliable without external dependencies.